



RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills

Department of Engineering

Software Requirements Specification

for

Fully Optimized Time Table Creation System

Version 1.0

Prepared by

Aditya Jagadale, Pratik Kawade, Punyam Shah

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies

3. Specific Requirements

- 3.1 External Interface Requirements
- 3.2 Functional Requirements
- 3.3 Non-Functional Requirements
- 3.4 Database Requirements

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed Software Requirements Specification (SRS) for the ****University Time Table Creation System****.

This document describes the functional and non-functional requirements for the system, which automates the creation of conflict-free university timetables. It is intended for use by developers, project managers, testers, and stakeholders involved in the development, deployment, and maintenance of the system.

1.2 Scope

The University Time Table Creation System is a web-based application designed to automate the process of generating, managing, and viewing university timetables. The system supports three types of users — ****Administrator****, ****Teacher****, and ****Student**** — each with role-specific dashboards and functionality.

Key capabilities include:

- Automated conflict-free timetable generation using a Constraint Satisfaction Problem (CSP) solver with backtracking
- Management of departments, courses, teachers, classrooms, and student batches
- Role-based dashboards for administrators, teachers, and students
- Lecture cancellation and restoration
- Teacher change request workflow
- Student elective course enrollment
- Daily and weekly timetable views
- Batch auto-splitting for large student groups

Out of Scope:

- Examination scheduling
- Fee management
- Student attendance tracking
- Mobile native application

1.3 Definitions, Acronyms, and Abbreviations

Academic Year: A period of formal instruction typically spanning twelve months, divided into semesters. This system assumes a maximum of four academic years for any program.

Administrator (Admin): A system user with the highest level of access privileges, responsible for managing all academic resources, generating timetables, and approving or rejecting change requests.

API (Application Programming Interface): A set of definitions and protocols for building and integrating application software. In this system, the API refers to the RESTful endpoints exposed by the server for client-server communication.

Batch: A subdivision of students within a department and year level, typically used for laboratory sessions where smaller group sizes are required. Batches are identified by section letters such as A, B, C.

Change Request: A formal request submitted by a teacher to modify an existing timetable entry, which may involve rescheduling to a different day and time, canceling a session, or requesting a room change.

Constraint Satisfaction Problem (CSP): A mathematical problem involving finding values for a set of variables that satisfy a collection of constraints. In this system, timetable generation is modeled as a CSP.

Course: An academic subject offered within a department, characterized by its code, credit hours, lecture and laboratory requirements, and assigned instructor.

CRUD: An acronym representing the four basic operations of persistent storage: Create, Read, Update, and Delete.

Department: An academic unit within the university responsible for offering courses and programs in a specific discipline.

Elective Course: A course that students may choose to enroll in based on their interests, as opposed to mandatory courses that all students in a program must complete.

Enrollment: The act of a student registering for an elective course, which affects which timetable entries are visible to that student.

Hard Constraint: A constraint that must be satisfied for a timetable to be valid. Violations of hard constraints result in conflicts that make the schedule unusable.

JSON (JavaScript Object Notation): A lightweight data interchange format used for storing and transmitting data. This system uses JSON files for data persistence.

Laboratory (Lab): A practical session for a course, typically requiring specialized facilities and smaller student groups. Lab sessions often span multiple consecutive time slots.

Lecture: A theoretical teaching session for a course, typically delivered in a classroom to all students in a batch or across batches.

MRV (Minimum Remaining Values): A heuristic used in constraint satisfaction problems that selects the variable with the fewest legal values

remaining. In this system, it prioritizes scheduling the most constrained sessions first.

REST (Representational State Transfer): An architectural style for designing networked applications, using standard HTTP methods for communication between client and server.

Semester: A half-year academic term. This system supports eight semesters across four years, with odd semesters in the first half of each year and even semesters in the second half.

Session: In the context of timetable generation, a session refers to a single schedulable unit derived from a course's weekly lecture or laboratory requirements.

Slot: A fixed time period during the day when classes can be scheduled. This system defines seven one-hour slots from 9:00 AM to 5:00 PM with a lunch break from 1:00 PM to 2:00 PM.

Soft Constraint: A preference that should be satisfied if possible but whose violation does not make the schedule invalid. Soft constraints are used to optimize schedule quality.

SPA (Single Page Application): A web application that interacts with the user by dynamically rewriting the current page rather than loading entire new pages from the server.

Student: A system user who can view their class schedule and manage elective course enrollments.

Teacher: A system user who can view their teaching schedule and submit change requests. Teachers are linked to courses and departments.

Timetable Entry: A single scheduled session in the timetable, specifying the course, teacher, classroom, day, time slot, and associated batch.

User: Any person who interacts with the system, classified into one of three roles: Administrator, Teacher, or Student.

1.4 References

This document references the following standards, guidelines, and resources:

IEEE Standard 830-1998: IEEE Recommended Practice for Software Requirements Specifications. This standard provides guidance on the content and qualities of a good software requirements specification.

IEEE Standard 29148-2018: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. This is the updated standard that supersedes IEEE 830 and provides comprehensive guidance on requirements engineering.

The Document Object Model (DOM) Level 3 Core Specification. World Wide Web Consortium. This specification is relevant to the client-side implementation using React.

ECMAScript 2021 Language Specification. ECMA International. This specification defines the JavaScript language used in both client and server implementations.

Hypertext Transfer Protocol (HTTP/1.1) Specification. RFC 7230-7235. Internet Engineering Task Force. This defines the communication protocol used between client and server.

JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259. Internet Engineering Task Force. This defines the data format used for storage and API communication.

React Documentation. Meta Platforms, Inc. This documentation covers the React library used for building the user interface.

Express.js Documentation. OpenJS Foundation. This documentation covers the Express framework used for building the server application.

Node.js Documentation. OpenJS Foundation. This documentation covers the Node.js runtime used for server-side JavaScript execution.

1.1 Overview of Document

This Software Requirements Specification is organized into seven major sections, each addressing a distinct aspect of the system requirements.

Section 1 :which you are currently reading, introduces the document by explaining its purpose, defining the scope of the system, establishing terminology, listing references, and providing this structural overview.

Section 2 : provides an overall description of the system, placing it in context relative to other systems and stakeholders. This section describes the product's position in the larger ecosystem, summarizes its major functions, characterizes the users who will interact with it, specifies the operating environment, identifies constraints on design and implementation, and states assumptions and dependencies.

Section 3: forms the core of this document, presenting detailed functional requirements organized by system modules. Each module is described in terms of its purpose, the specific requirements it must satisfy, the inputs it accepts, the outputs it produces, and the business rules that govern its behavior. This section covers authentication, resource management, timetable generation, change requests, and enrollment management.

Section 4: specifies external interface requirements, describing how the system interacts with users through its user interface, with hardware devices, with other software systems, and over communication networks.

Section 5: presents non-functional requirements that specify quality attributes the system must exhibit. These include performance

characteristics, security measures, reliability expectations, availability targets, maintainability considerations, and portability requirements.

Section 6: details data requirements, providing a comprehensive data dictionary that defines all data entities, their attributes, and relationships. This section also specifies data integrity constraints and data retention policies.

Section 7: contains appendices with supplementary technical information, including a detailed specification of the timetable generation algorithm, a reference list of business rules, and a catalog of error codes and messages.

2 Overall Description

2.1 Product Perspective

The University Timetable Creation and Management System is designed as a self-contained web application that operates independently without requiring integration with external systems. It is intended to serve as the primary tool for academic scheduling within an educational institution, replacing manual spreadsheet-based processes or legacy scheduling software.

The system follows a client-server architecture with clear separation between the presentation layer, application logic, and data storage. The client component is a single-page application that runs in web browsers and communicates with the server through RESTful API endpoints. The server component handles business logic, data validation, and persistence operations.

From a deployment perspective, the system operates as a single instance serving all users within an institution. It does not currently support multi-tenant deployment where multiple institutions share the same system instance with isolated data. However, the modular architecture would facilitate such an extension if required in the future.

The system does not integrate with external student information systems, human resource systems, or learning management systems. All data

regarding departments, courses, teachers, students, and classrooms must be entered directly into the system. This independence simplifies deployment but requires manual data synchronization if the institution maintains parallel systems.

The user interface is designed for desktop and laptop computers with standard web browsers. While the responsive design provides basic functionality on tablet devices, the system is not optimized for mobile phone screens, and certain features may be difficult to use on smaller displays.

The system assumes a stable network connection between clients and the server. It does not implement offline functionality or local data caching beyond standard browser behavior. Users must have network access to perform any system operations.

2.2 Product Functions

#	Function	Description
F1	User Authentication	Role-based login for Admin, Teacher, and Student
F2	Department Management	CRUD operations for university departments
F3	Course Management	CRUD for courses with department, semester, and teacher association
F4	Teacher Management	CRUD for teachers with department and course assignment

F5	Classroom Management	CRUD for lecture halls and laboratory rooms
F6	Batch Management	CRUD for student batches with auto-splitting
F7	Timetable Generation	Automated conflict-free scheduling using CSP algorithm
F8	Timetable Viewing	Daily and weekly views for all user roles
F9	Lecture Management	Cancel/restore individual lectures (Teacher)
F10	Change Request Workflow	Teachers submit requests; Admin approves/rejects
F11	Elective Enrollment	Students enroll/unenroll in elective courses
F12	Admin Dashboard	Overview statistics and system-wide management

2.3 User Characteristics

Role	Description	Default Credentials
Administrator	Full access to all modules. Manages departments, courses, teachers, classrooms, batches. Generates timetables. Reviews change requests.	admin / admin123

Teacher	Views personal daily/weekly timetable. Cancels/restores lectures. Submits change requests.	teacher / teacher123
Student	Views department timetable. Views enrolled courses. Enrolls/unenrolls in electives.	student1 / student123

2.4 Constraints

#	Constraint
1	Built using React.js (v18.x) front-end and Node.js/Express.js (v4.x) back-end
2	JSON file-based storage; no external database server required
3	Accessible via modern web browsers (Chrome, Firefox, Edge, Safari)
4	Runs on single server (localhost:5000) serving API and React client
5	Working days: Monday through Saturday (6 days)
6	Simple username/password authentication (academic project scope)

3. Specific Requirements

3.1 User Interface Requirements

UI-ID	Requirement	Description
UI-01	Login Page	Form with username/password fields, role-based redirection on success
UI-02	Admin Dashboard	Statistics overview: counts of departments, courses, teachers, classrooms, batches
UI-03	Admin Sidebar	Links to Dashboard, Timetable, Courses, Teachers, Classrooms, Departments, Batches, Requests, View All
UI-04	Teacher Dashboard	Daily timetable as default with navigation to weekly view, lecture management, change requests
UI-05	Student Dashboard	Daily timetable as default with navigation to weekly view, courses, electives
UI-06	Timetable Grid	Responsive grid: days as rows, time slots as columns, color-coded lecture/lab entries
UI-07	CRUD Forms	Modal/inline forms for creating/editing all entities
UI-08	Responsive Design	Must render correctly on screens ≥ 1024 px width

3.2 Software Interface Requirements

Software	Purpose	Version
Node.js	Server runtime environment	v14+
Express.js	HTTP server framework	v4.18+
React.js	Front-end UI library	v18.2+
React Router	Client-side routing	v6.22+
Axios	HTTP client for API calls	v1.6+
CORS	Cross-Origin Resource Sharing middleware	v2.8+

3.3 Functional Requirements — Authentication

REQ-ID	Requirement	Priority
FR-AUTH-01	The system shall provide a login page with username and password fields.	High
FR-AUTH-02	The system shall authenticate users against stored credentials.	High
FR-AUTH-03	Upon successful login, redirect to role-specific dashboard (Admin→/admin, Teacher→/teacher, Student→/student).	High

FR-AUTH-04	Display error message for invalid credentials.	High
FR-AUTH-05	Prevent unauthorized access to role-specific routes using route guards.	High
FR-AUTH-06	Persist user session in the browser context.	Medium

3.4 Functional Requirements — Department Management

REQ-ID	Requirement	Priority
FR-DEPT-01	Admin can create a department with: name, code, HOD, description, years.	High
FR-DEPT-02	Admin can view a list of all departments.	High
FR-DEPT-03	Admin can update department details.	High
FR-DEPT-04	Admin can delete a department.	High
FR-DEPT-05	Each department has a unique system-generated ID.	High

3.5 Functional Requirements — Course Management

REQ-ID	Requirement	Priority
FR-CRS-01	Admin can create a course with: name, code, department(s), year, semester, weekly lectures, weekly labs, lecture duration, lab duration, type (core/elective), teacher.	High
FR-CRS-02	Admin can view courses filtered by department, year, and semester.	High
FR-CRS-03	Admin can update course details.	High
FR-CRS-04	Admin can delete a course.	High
FR-CRS-05	A course may belong to multiple departments (shared courses).	Medium
FR-CRS-06	System displays assigned teacher name alongside each course.	Medium
FR-CRS-07	Numeric fields are automatically parsed to integers.	Low

3.6 Functional Requirements — Teacher Management

REQ-ID	Requirement	Priority
FR-TCH-01	Admin can create a teacher with: name, title, email, phone, department(s), specialization, office, bio, courses.	High
FR-TCH-02	Creating a teacher automatically creates a linked user account (username from email prefix, password: teacher123).	High
FR-TCH-03	Admin can view teachers filtered by department.	High
FR-TCH-04	Admin can update teacher details.	High
FR-TCH-05	Updating teacher courses auto-updates teacherId on respective courses (assign/unassign).	High
FR-TCH-06	Deleting a teacher unassigns from all courses and deletes linked user account.	High
FR-TCH-07	A teacher may be assigned to multiple departments.	Medium

3.7 Functional Requirements — Classroom Management

REQ-ID	Requirement	Priority
FR-CLR-01	Admin can create a classroom with: name, type (lecture/lab), capacity.	High
FR-CLR-02	Admin can view classrooms filtered by type.	High
FR-CLR-03	Admin can update classroom details.	High
FR-CLR-04	Admin can delete a classroom.	High

3.8 Functional Requirements — Batch Management

REQ-ID	Requirement	Priority
FR-BCH-01	Admin can create a batch with: name, section, department, year, student count.	High
FR-BCH-02	Admin can view batches filtered by department and year.	High
FR-BCH-03	Admin can update batch details.	High
FR-BCH-04	Admin can delete a batch.	High

FR-BCH-05	Auto-split: given total students and max batch size, auto-create batches (A, B, C...) with even distribution.	High
FR-BCH-06	Auto-split removes existing batches for same department+year before creating new ones.	Medium

3.9 Functional Requirements — Timetable Generation

REQ-ID	Requirement	Priority
FR-TT-01	Admin can generate a timetable for a specific department and semester.	High
FR-TT-02	Uses CSP algorithm with backtracking for conflict-free scheduling.	High
FR-TT-03a	Hard Constraint: No teacher assigned to two classes at the same time.	High
FR-TT-03b	Hard Constraint: No classroom assigned to two classes at the same time.	High
FR-TT-03c	Hard Constraint: No student group has two classes at the same time.	High
FR-TT-03d	Hard Constraint: Lab sessions must not cross lunch break.	High
FR-TT-03e	Hard Constraint: Room type must match session type.	High

FR-TT-04a	Soft Constraint: Spread course lectures across different days.	Medium
FR-TT-04b	Soft Constraint: Balance daily class load per student group.	Medium
FR-TT-04c	Soft Constraint: Penalize 3+ consecutive classes.	Medium
FR-TT-04d	Soft Constraint: Lectures prefer morning; labs prefer afternoon.	Medium
FR-TT-04e	Soft Constraint: Distribute teacher load evenly across days.	Medium
FR-TT-04f	Soft Constraint: Prefer smallest room that fits batch size.	Medium
FR-TT-05	Uses MRV heuristic to schedule most constrained sessions first.	Medium
FR-TT-06	Supports backtracking (up to 50 iterations) to resolve failures.	Medium
FR-TT-07	Reports errors for sessions that could not be placed.	High
FR-TT-08	Supports generating timetable for a specific batch.	Medium
FR-TT-09	Generate timetables for all semesters (1–8) of a department at once.	Medium

FR-TT-10	Delete all timetable entries for a specific department+semester.	Medium
-----------------	---	---------------

3.10 Functional Requirements — Timetable Viewing

REQ-ID	Requirement	Priority
FR-TV-01	Admin can view timetables for any department and semester.	High
FR-TV-02	Admin can view all timetable entries system-wide.	Medium
FR-TV-03	Teachers can view personal daily timetable.	High
FR-TV-04	Teachers can view personal weekly timetable.	High
FR-TV-05	Students can view department daily timetable.	High
FR-TV-06	Students can view department weekly timetable.	High
FR-TV-07	Entries sorted by day order (Mon→Sat) then by time slot.	Medium
FR-TV-08	Each entry displays: course name, code, teacher, classroom, time slot, type.	High

3.11 Functional Requirements — Lecture Management (Teacher)

REQ-ID	Requirement	Priority
FR-LM-01	Teacher can cancel a scheduled lecture (status → "cancelled").	High
FR-LM-02	Teacher can restore a cancelled lecture (status → "active").	High
FR-LM-03	Cancelled lectures remain in timetable but visually distinguished.	Medium

3.12 Functional Requirements — Change Requests

REQ-ID	Requirement	Priority
FR-CR-01	Teacher can submit a change request with details and reason.	High
FR-CR-02	Each request has status: "pending", "approved", or "rejected".	High
FR-CR-03	Admin can view all change requests with teacher names.	High
FR-CR-04	Admin can approve or reject a change request.	High
FR-CR-05	Change requests are timestamped at creation.	Low

3.13 Functional Requirements — Elective Enrollment (Student)

REQ-ID	Requirement	Priority
FR-EL-01	Student can view available elective courses.	Medium
FR-EL-02	Student can enroll in an elective course.	Medium
FR-EL-03	Student can unenroll from an elective course.	Medium
FR-EL-04	System prevents duplicate enrollments (same user + same course).	Medium
FR-EL-05	Each enrollment is timestamped.	Low

3.14 Non-Functional Requirements — Performance

REQ-ID	Requirement
NFR-PERF-01	Timetable generation for single department+semester shall complete within 10 seconds.
NFR-PERF-02	API response time for CRUD operations shall be under 500ms.
NFR-PERF-03	System shall support concurrent access by at least 50 users.
NFR-PERF-04	React client shall render initial page within 3 seconds.

3.15 Non-Functional Requirements — Reliability

REQ-ID	Requirement
NFR-REL-01	System shall not lose data during normal operations.
NFR-REL-02	Handle server errors gracefully with appropriate HTTP status codes.
NFR-REL-03	Timetable generator reports unplaceable sessions as errors rather than crashing.

3.16 Non-Functional Requirements — Usability

REQ-ID	Requirement
NFR-USE-01	Modern, dark-themed UI with clear navigation.
NFR-USE-02	Role-specific dashboards present only relevant information.
NFR-USE-03	Forms include input validation and error feedback.
NFR-USE-04	Timetable grid uses color coding for lecture vs lab sessions.

3.17 Non-Functional Requirements — Security

REQ-ID	Requirement
NFR-SEC-01	Role-based access control (RBAC) with three roles: admin, teacher, student.
NFR-SEC-02	Route guards prevent unauthorized access to restricted pages.
NFR-SEC-03	Password fields excluded from API responses.

3.18 Non-Functional Requirements — Maintainability

REQ-ID	Requirement
NFR-MNT-01	Modular architecture with separate client and server directories.
NFR-MNT-02	Generic database abstraction layer (db.js) to enable future migration to relational DB.
NFR-MNT-03	Scheduling algorithm encapsulated in separate module (generator.js).